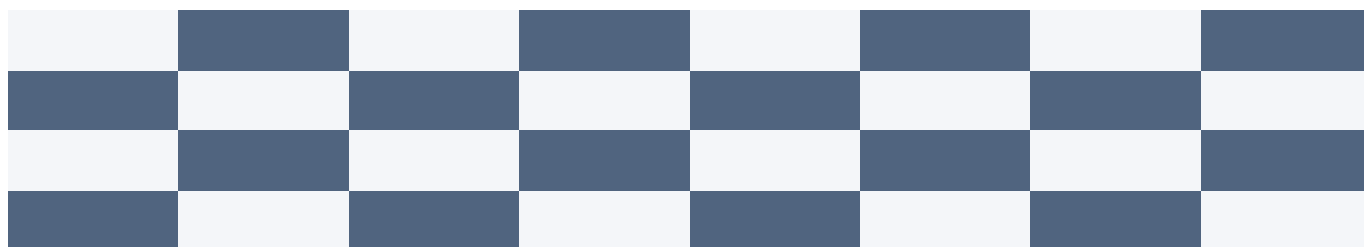


# JOGO DE XADREZ

## Tutorial Completo do Código



Motor completo de regras, Inteligência Artificial e quatro câmeras

# Sumário

- 1. Arquitetura geral do projeto**
- 2. index.html: estrutura da interface**
- 3. estilo.css: layout e responsividade**
4. motor.js: modelo de estado do xadrez
5. Criação das peças e movimentos
6. Geração e filtragem de movimentos legais
7. Peões, en passant e promoção
8. Cavalos, peças deslizantes e rei
9. Roque e direitos de roque
10. Detecção de casas atacadas e xeque
11. Aplicação de movimentos e histórico
12. Resultados e regras de empate
13. Repetição, 50 e 75 lances
14. ia.js: arquitetura da Inteligência Artificial
15. Nível Fácil
16. Minimax e poda Alpha-Beta
17. Função de avaliação da IA
18. Ordenação de movimentos e anti-ciclo
19. renderizador3d.js: Canvas e câmeras
20. Projeção do tabuleiro e clique nas casas
21. Desenho das peças 3D
22. app.js: integração da aplicação
23. Seleção e execução de jogadas
24. Fluxo da IA e controle de tempo
25. Promoção e eventos especiais
26. Finalização, empate e desistência
27. Sons e controles do navegador
28. Persistência, ranking e estatísticas
29. Mapa das principais funções
30. Fluxo completo de uma jogada
31. Decisões de projeto e extensão

# 1. Arquitetura geral do projeto

O projeto foi dividido por responsabilidade. Essa separação reduz o acoplamento entre as regras, a Inteligência Artificial, a interface e a renderização. O HTML define os elementos visuais; o CSS cuida do layout; motor.js representa as regras; ia.js escolhe jogadas; renderizador3d.js desenha o tabuleiro e as peças; app.js conecta tudo e controla o fluxo da partida.

A aplicação não depende de bibliotecas externas. Toda a lógica de xadrez, busca da IA e renderização em Canvas foi implementada no próprio projeto.

| Arquivo              | Responsabilidade principal                | Linhas |
|----------------------|-------------------------------------------|--------|
| index.html           | Estrutura da interface, menus e painéis   | 1-1019 |
| css/estilo.css       | Layout, temas, responsividade e câmeras   | 1-4364 |
| js/motor.js          | Regras e estado completo do xadrez        | 1-5232 |
| js/ia.js             | IA, Minimax, poda e avaliação             | 1-2170 |
| js/renderizador3d.js | Canvas, projeção, câmeras e peças 3D      | 1-8027 |
| js/app.js            | Interface, sons, fluxo, ranking e eventos | 1-7188 |

## 2. index.html: estrutura da interface

O arquivo index.html funciona como a estrutura semântica da aplicação. Ele possui a tela de configuração, seletores de modo, cor, dificuldade e câmera, botões de início e telas auxiliares. Na área de partida ficam o seletor de câmera, o tabuleiro/Canvas, painéis de jogador, histórico, capturas e ações como reivindicar empate, oferecer empate, desistir e voltar ao menu.

Os elementos recebem identificadores que app.js consulta com document.getElementById. Dessa maneira, o JavaScript não precisa criar toda a interface do zero; ele atualiza elementos previamente definidos no HTML.

## 3. estilo.css: layout e responsividade

O CSS concentra a identidade visual, dimensões do tabuleiro, painéis, botões, modais, estados de seleção e regras responsivas. Também existem estilos relacionados aos quatro enquadramentos, ao Canvas e aos elementos que precisam permanecer legíveis em diferentes tamanhos de navegador.

A responsividade é importante porque o renderizador calcula seu tamanho a partir da área disponível. O CSS define o espaço; o JavaScript mede esse espaço e adapta o desenho.

## 4. motor.js: modelo de estado do xadrez

A classe XadrezMotor, em motor.js, é o núcleo das regras. O construtor começa na linha 290 e prepara propriedades como tabuleiro, turno, direitos de roque, possibilidade de en passant, histórico e estruturas de repetição. O método reiniciar(), a partir da linha 408, restaura a posição inicial.

O método clonar(), a partir da linha 658, cria uma cópia independente do estado. Essa operação é fundamental para a IA, que precisa simular movimentos sem alterar a partida real.

| Elemento      | Linhas aproximadas | Função                                    |
|---------------|--------------------|-------------------------------------------|
| constructor() | 290-407            | Inicializa o estado.                      |
| reiniciar()   | 408-657            | Monta a posição inicial e zera controles. |
| clonar()      | 658-874            | Copia o estado para simulações.           |
| pecaEm()      | 897-924            | Lê uma peça em determinada casa.          |

## 5. Criação das peças e movimentos

A função criarPeca(), nas linhas 142-169 de motor.js, padroniza o objeto que representa cada peça. criarMovimento(), nas linhas 170-259, monta uma estrutura de movimento contendo origem, destino, peça e opções especiais.

Essa representação única permite que movimentos comuns, capturas, roques, en passant e promoções passem pelo mesmo fluxo de aplicação. Em vez de espalhar várias formas de dados, o motor trabalha com um formato consistente.

## 6. Geração e filtragem de movimentos legais

movimentosLegais(), a partir da linha 925, produz todos os movimentos permitidos para uma cor. movimentosDaCasa(), na linha 1016, restringe a consulta a uma origem específica, o que é usado pela interface quando o jogador seleciona uma peça.

O motor primeiro gera movimentos pseudo-legais e depois elimina os que deixam o próprio rei em xeque. Essa separação simplifica a implementação: cada peça cuida da geometria do seu movimento, enquanto a validação final cuida da segurança do rei.

| Método                      | Linhas    | Papel                                          |
|-----------------------------|-----------|------------------------------------------------|
| movimentosLegais()          | 925-1015  | Lista todos os movimentos legais da cor.       |
| movimentosDaCasa()          | 1016-1045 | Lista destinos legais de uma peça selecionada. |
| explicarMovimentoInvalido() | 1046-1467 | Produz explicação em português para            |

|                                        |           |                                           |
|----------------------------------------|-----------|-------------------------------------------|
|                                        |           | tentativa inválida.                       |
| <code>_movimentosPseudoLegais()</code> | 1468-1690 | Gera candidatos antes do filtro de xeque. |

## 7. Peões, en passant e promoção

`_movimentosPeao()`, linhas 1691-2077, concentra as particularidades da peça mais complexa em regras especiais: avanço simples, avanço duplo inicial, captura diagonal, en passant e geração de opções de promoção.

O estado guarda a casa relevante para en passant apenas pelo período em que a captura é permitida. Na promoção, o movimento carrega a informação da peça escolhida, permitindo que a aplicação substitua o peão no destino.

## 8. Cavalos, peças deslizantes e rei

`_movimentosCavalo()`, linhas 2078-2210, usa os oito deslocamentos em L e não precisa verificar casas intermediárias. `_movimentosDeslizantes()`, linhas 2211-2377, é reutilizado para torre, bispo e dama percorrendo uma direção até encontrar a borda ou uma peça.

`_movimentosRei()`, linhas 2378-2539, trata os passos de uma casa e agrega os candidatos de roque. O filtro legal posterior garante que o rei não termine atacado.

## 9. Roque e direitos de roque

`_movimentosRoque()`, linhas 2540-2813, verifica direitos registrados, presença da torre, casas vazias e segurança das casas percorridas pelo rei. `_atualizarDireitosRoque()`, linhas 4055-4301, remove os direitos quando rei ou torre se movem ou quando uma torre relevante é capturada.

Separar geração e manutenção dos direitos impede que um rei retorne a casa inicial e volte a poder rocar indevidamente.

## 10. Detecção de casas atacadas e xeque

`casaAtacada()`, linhas 2814-3355, responde se uma casa é controlada pela cor adversária. `encontrarRei()`, linhas 3356-3440, localiza o rei da cor desejada. `estaEmXeque()`, linhas 3441-3488, combina as duas operações.

Essas funções são usadas em diversas regras: filtro de movimentos legais, roque, detecção de xeque e resultado final.

## 11. Aplicação de movimentos e histórico

`aplicarMovimento()`, a partir da linha 3489, é a entrada pública para executar uma jogada válida.

`_aplicarSemValidar()`, a partir da linha 3568, efetua as alterações concretas no tabuleiro e é útil em simulações internas.

O movimento atualiza captura, promoção, en passant, deslocamento da torre no roque, contador de meios-lances, turno, histórico e chave de repetição. Centralizar essas atualizações evita divergência entre jogadas do usuário e da IA.

## 12. Resultados e regras de empate

resultado(), linhas 4302-4411, verifica se a partida deve terminar. A avaliação considera ausência de movimentos legais, xeque-mate, afogamento, material insuficiente e empates automáticos por repetição ou contagem de lances.

podeReivindicarEmpateRepeticao(), podeReivindicarEmpate50Lances() e motivosEmpateReivindicavel() separam as condições que dependem de uma ação do jogador.

| Método                           | Linhas    |
|----------------------------------|-----------|
| resultado()                      | 4302-4411 |
| podeReivindicarEmpateRepeticao() | 4412-4440 |
| podeReivindicarEmpate50Lances()  | 4441-4461 |
| motivosEmpateReivindicavel()     | 4462-4513 |
| materialInsuficiente()           | 4514-4682 |

## 13. Repetição, 50 e 75 lances

\_chavePosicao(), linhas 4683-4803, cria uma representação da posição usada na contagem de repetições. O cálculo inclui informações que alteram a legalidade futura, como turno, direitos de roque e en passant legalmente disponível.

\_registrarPosicao(), linhas 5021-5231, incrementa a ocorrência da configuração atual. O contador de meios-lances é atualizado na aplicação dos movimentos e serve para as regras de 50 e 75 lances.

## 14. ia.js: arquitetura da Inteligência Artificial

A classe XadrezIA está em ia.js. O construtor inicia na linha 65 e recebe o nível. escolherMovimento(), a partir da linha 98, é a entrada principal. Ela consulta os movimentos legais, normaliza o nome do nível e direciona o processamento.

No Intermediário a profundidade principal é 2; no Avançado, 3. A busca utiliza cópias do motor, aplica movimentos hipotéticos e avalia a posição resultante sem modificar a partida visível.

## 15. Nível Fácil

\_escolherFacil(), linhas 386-576, implementa o comportamento do nível mais acessível. Em vez de explorar a árvore com a mesma profundidade dos níveis superiores, ele escolhe usando critérios mais leves, permitindo maior variação e erros estratégicos.

A existência de um caminho separado para o nível Fácil evita gastar processamento desnecessário e cria uma diferença real de desafio.

## 16. Minimax e poda Alpha-Beta

`_minimax()`, linhas 577-1026, implementa a busca adversarial. Em um nível, a IA tenta maximizar sua avaliação; no seguinte, assume que o adversário escolherá a resposta que minimiza esse valor. O processo continua até a profundidade definida ou até uma posição terminal.

Os parâmetros alfa e beta armazenam limites conhecidos. Quando um ramo não pode mais melhorar o resultado já encontrado, a poda Alpha-Beta interrompe sua exploração. Isso reduz a quantidade de posições avaliadas sem alterar a decisão Minimax para a mesma ordem e profundidade.

## 17. Função de avaliação da IA

`avaliar()`, linhas 1508-1723, converte uma posição em um valor numérico do ponto de vista da cor da IA. A avaliação combina material e bônus posicionais. `_bonusPosicional()`, linhas 1724-1950, valoriza a colocação das peças em casas mais adequadas ao seu papel.

Uma função de avaliação é necessária porque a busca normalmente termina antes do fim real da partida. O número estimado representa quão favorável é a posição naquele ponto da árvore.

## 18. Ordenação de movimentos e anti-ciclo

`_ordenarMovimentos()`, linhas 1951-2170, coloca candidatos promissores em ordem que favorece a poda. A classe ainda possui `deveReivindicarEmpate()`, ajustes anti-ciclo e verificações para evitar desfazer repetidamente o próprio movimento ou mover a mesma peça sem ganho.

Esses mecanismos deixam o comportamento mais natural e reduzem loops que poderiam surgir em avaliadores de profundidade limitada.

## 19. renderizador3d.js: Canvas e câmeras

`renderizador3d.js` implementa a classe responsável pelo Canvas. O construtor inicia na linha 35. `definirCamera()`, linha 264, troca o enquadramento; `renderizar()`, linha 304, recebe o estado visual; `_ajustarCanvas()`, linha 378, sincroniza resolução e área disponível.

`_configuracao()`, a partir da linha 565, fornece parâmetros específicos para cada câmera. A Câmera Superior usa uma projeção mais direta, enquanto as câmeras de perspectiva transformam as coordenadas das casas para produzir profundidade aparente.

## 20. Projeção do tabuleiro e clique nas casas

`_projetar()`, linhas 802-929, transforma coordenadas lógicas do tabuleiro em coordenadas de tela. As funções `_viewRow()`, `_viewCol()`, `_actualRow()` e `_actualCol()` convertem a orientação de acordo com o lado da câmera.

Para detectar cliques, o renderizador guarda a geometria das casas projetadas. `_pontoNoPoligono()` testa se o cursor está dentro de uma casa; `_casaNoPonto()`, a partir da linha 7757, retorna a casa correspondente e `_clicar()`, linha 7875, encaminha a seleção para `app.js`.

## 21. Desenho das peças 3D

`_desenharPecas()`, linha 3322, percorre as peças visíveis e chama `_desenharPeca()`, linha 3744. O desenho não usa imagens prontas: cada modelo é construído com Canvas, gradientes e perfis geométricos.

Há rotinas especializadas para peão, torre, bispo, dama, rei e cavalo. Funções auxiliares como `_gradienteMaterial()`, `_basePeca()`, `_corpoPerfil()`, `_anel()` e `_esfera()` reutilizam componentes comuns para manter consistência visual.

| Peça / componente                 | Início aproximado |
|-----------------------------------|-------------------|
| <code>_peao()</code>              | 5323              |
| <code>_torre()</code>             | 5418              |
| <code>_bispo()</code>             | 5796              |
| <code>_dama()</code>              | 6082              |
| <code>_rei()</code>               | 6413              |
| <code>_cavalo()</code>            | 6665              |
| <code>_gradienteMaterial()</code> | 4233              |
| <code>_basePeca()</code>          | 4344              |

## 22. app.js: integração da aplicação

`app.js` é a camada de orquestração. Ele referencia `XadrezMotor`, `XadrezIA` e `RenderizadorXadrez3D`, lê os controles da página, administra sons, estados de tela, modais e armazenamento local.

`criarTabuleiroDOM()`, linha 2227, prepara a representação interativa usada pela aplicação. `renderizarTabuleiro()`, linha 2502, sincroniza seleção, destaques e renderizador 3D. `renderizarTudo()`, linha 3293, atualiza tabuleiro e painéis em conjunto.

## 23. Seleção e execução de jogadas

`clicarCasa()`, linhas 3350-3688, recebe a casa clicada. Ele verifica se existe partida, de quem é o turno, se a casa possui uma peça selecionável e quais destinos são legais. Se houver mais de uma opção para o mesmo destino por causa de promoção, o fluxo chama `abrirPromocao()`.

`executarMovimento()`, linhas 3916-4181, aplica o movimento no motor, aciona efeitos, atualiza a interface, verifica o resultado e agenda a próxima jogada quando ela pertence à IA.

## 24. Fluxo da IA e controle de tempo

`agendarIA()`, linhas 4303-4397, programa a próxima ação automática respeitando o estado atual da partida. `iniciarCalculoIA()`, linhas 4398-4605, chama a IA para escolher um movimento e garante que o retorno ainda pertence à mesma partida antes de aplicar o lance.

Esse cuidado é importante porque o usuário pode voltar ao menu, pausar ou iniciar outra partida enquanto existe uma ação programada. Identificar a partida evita que um cálculo antigo seja aplicado ao estado novo.

## 25. Promoção e eventos especiais

`abrirPromocao()`, linhas 3689-3915, apresenta as opções de promoção quando um peão alcança a última fileira. Depois que uma opção é escolhida, o movimento correspondente segue para `executarMovimento()`.

`mostrarAvisoCaptura()`, linhas 4182-4302, controla avisos temporários de captura. Sons específicos são acionados de acordo com o tipo do movimento, incluindo captura, roque, promoção e xeque.

## 26. Finalização, empate e desistência

`iniciarPartida()`, linhas 4606-4924, coleta configurações, cria o motor, instancia IAs conforme o modo, prepara a câmera, limpa estados anteriores e inicia o fluxo. `traduzirResultado()`, linha 4925, converte códigos internos em textos; `finalizarResultado()`, linha 5143, registra e apresenta o resultado.

`reivindicarEmpate()`, linha 5476, consulta os motivos permitidos pelo motor. `oferecerEmpate()`, linha 5627, implementa a negociação com a IA. `desistir()`, linha 5844, confirma a ação e finaliza com vitória adversária.

## 27. Sons e controles do navegador

O mapa de sons é criado em `app.js` e associa nomes lógicos a arquivos WAV. `tocarSom()`, linha 1031, reproduz o efeito apenas quando o som está habilitado. `alternarSom()`, linha 6576, muda o estado e é usado pelo botão e pela tecla F8.

`cameraAplicar()`, linha 1690, envia a câmera escolhida ao renderizador e atualiza os controles. Os eventos de teclado também mapeiam 1 a 4 para câmeras e F8 para áudio.

## 28. Persistência, ranking e estatísticas

A persistência é encapsulada em uma classe de armazenamento dentro de `app.js`. `_ler()`, linha 433, recupera JSON do `localStorage`; `_salvar()`, linha 531, grava os dados; `registrarPartida()`, linha 570, atualiza o perfil e adiciona o resultado ao histórico.

`estatisticas()`, linha 805, retorna os dados de um jogador; `ranking()`, linha 846, ordena os perfis; `historico()`, linha 890, fornece as partidas registradas. O sistema soma 3 pontos por vitória, 1 por empate e 0 por derrota.

## 29. Mapa das principais funções

A tabela abaixo funciona como um índice técnico para localizar rapidamente os principais pontos do projeto. Os intervalos correspondem ao arquivo fornecido.

| Arquivo           | Função / método                  | Linhas    | Responsabilidade                |
|-------------------|----------------------------------|-----------|---------------------------------|
| motor.js          | XadrezMotor.constructor()        | 290-407   | Cria o estado do jogo.          |
| motor.js          | reiniciar()                      | 408-657   | Restaura a posição inicial.     |
| motor.js          | clonar()                         | 658-874   | Copia o estado para simulação.  |
| motor.js          | movimentosLegais()               | 925-1015  | Gera movimentos legais.         |
| motor.js          | movimentosDaCasa()               | 1016-1045 | Destinos de uma peça.           |
| motor.js          | explicarMovimentoInvalido()<br>) | 1046-1467 | Explica tentativa inválida.     |
| motor.js          | _movimentosPeao()                | 1691-2077 | Peão, en passant e promoção.    |
| motor.js          | _movimentosRoque()               | 2540-2813 | Valida roques.                  |
| motor.js          | casaAtacada()                    | 2814-3355 | Detecta controle de casa.       |
| motor.js          | estaEmXeque()                    | 3441-3488 | Detecta xeque.                  |
| motor.js          | aplicarMovimento()               | 3489-3567 | Entrada para executar lance.    |
| motor.js          | resultado()                      | 4302-4411 | Determina resultado da partida. |
| motor.js          | materialInsuficiente()           | 4514-4682 | Detecta empate por material.    |
| ia.js             | escolherMovimento()              | 98-385    | Entrada principal da IA.        |
| ia.js             | _escolherFacil()                 | 386-576   | Lógica do nível Fácil.          |
| ia.js             | _minimax()                       | 577-1026  | Busca Minimax com poda.         |
| ia.js             | deveReivindicarEmpate()          | 1027-1138 | Decide reivindicação.           |
| ia.js             | avaliar()                        | 1508-1723 | Avalia uma posição.             |
| ia.js             | _bonusPosicional()               | 1724-1950 | Bônus de posicionamento.        |
| ia.js             | _ordenarMovimentos()             | 1951-2170 | Ordena candidatos.              |
| renderizador3d.js | definirCamera()                  | 264-303   | Troca câmera.                   |
| renderizador3d.js | renderizar()                     | 304-342   | Recebe estado visual.           |
| renderizador3d.js | _projetar()                      | 802-929   | Projeta coordenadas no          |

|                   |                       |           |                                |
|-------------------|-----------------------|-----------|--------------------------------|
|                   |                       |           | Canvas.                        |
| renderizador3d.js | _desenharTabuleiro()  | 2103-3099 | Desenha casas e destaques.     |
| renderizador3d.js | _desenharPecas()      | 3322-3653 | Desenha conjunto de peças.     |
| renderizador3d.js | _desenharPeca()       | 3744-4232 | Despacha o modelo da peça.     |
| renderizador3d.js | _casaNoPonto()        | 7757-7874 | Converte clique em casa.       |
| app.js            | tocarSom()            | 1031-1278 | Reproduz efeitos sonoros.      |
| app.js            | cameraAplicar()       | 1690-1887 | Aplica câmera.                 |
| app.js            | renderizarTabuleiro() | 2502-2860 | Atualiza tabuleiro.            |
| app.js            | clicarCasa()          | 3350-3688 | Processa seleção humana.       |
| app.js            | abrirPromocao()       | 3689-3915 | Escolha de promoção.           |
| app.js            | executarMovimento()   | 3916-4181 | Executa e trata um lance.      |
| app.js            | agendarIA()           | 4303-4397 | Agenda turno automático.       |
| app.js            | iniciarCalculoIA()    | 4398-4605 | Calcula lance da IA.           |
| app.js            | iniciarPartida()      | 4606-4924 | Configura e inicia partida.    |
| app.js            | finalizarResultado()  | 5143-5475 | Finaliza e registra resultado. |
| app.js            | reivindicarEmpate()   | 5476-5626 | Processa reivindicação.        |
| app.js            | oferecerEmpate()      | 5627-5843 | Oferta de empate contra IA.    |
| app.js            | desistir()            | 5844-5936 | Processa desistência.          |
| app.js            | ranking()             | 6275-6374 | Exibe ranking.                 |
| app.js            | alternarSom()         | 6576-6674 | Liga/desliga áudio.            |

## 30. Fluxo completo de uma jogada

1. O usuário clica em uma casa. renderizador3d.js converte o ponto da tela na casa lógica e chama o callback configurado pela aplicação.
2. clicarCasa() consulta movimentosDaCasa() no motor. O motor gera candidatos e remove os que deixariam o rei em xeque.
3. Ao escolher um destino, executarMovimento() chama aplicarMovimento(). O motor altera o estado, registra histórico, atualiza roque, en passant, contadores e repetição.

4. `app.js` toca os sons apropriados e chama `renderizarTudo()`. O renderizador recebe o novo estado e redesenha tabuleiro, peças e destaques.
5. `resultado()` verifica xeque-mate, afogamento e empates. Se a partida continuar e o próximo turno for da IA, `agendarIA()` inicia a próxima decisão.
6. A IA clona o estado, explora movimentos hipotéticos e retorna um movimento. O mesmo `executarMovimento()` é usado, garantindo que usuário e IA passem pelo mesmo motor de regras.

## 31. Decisões de projeto e extensão

A principal decisão arquitetural é manter o motor independente da interface. Isso permite testar regras e simular posições sem depender do Canvas. A IA recebe o mesmo tipo de estado usado na partida real, e o renderizador apenas representa os dados que recebe.

Outra decisão importante é desenhar as peças em Canvas em vez de depender de modelos ou imagens externas. Isso torna o projeto autocontido e permite quatro câmeras com o mesmo conjunto de peças.

Para expandir o projeto, novos recursos podem ser adicionados preservando essa separação. Exemplos: relógio de xadrez pode ser tratado em `app.js`; novos critérios de avaliação entram em `ia.js`; importação/exportação de posições pertence ao motor e à camada de interface; novos estilos de câmera ficam no renderizador.

- Mantenha regras oficiais dentro de `motor.js`.
- Mantenha critérios de decisão automática dentro de `ia.js`.
- Mantenha desenho e projeção dentro de `renderizador3d.js`.
- Mantenha DOM, áudio, persistência e fluxo de telas dentro de `app.js`.
- Ao adicionar um recurso, reutilize `aplicarMovimento()` para não criar caminhos de jogada inconsistentes.